MATH-454: Parallel and high-performance computing

Series 8

Author:

Mattia Barbiere (387974)

Professor:

Pablo Antolin

Spring Semester - 2025

1 From CBLAS to CUDA

By simple inspection I noticed that the functions that had to be parallelized where all the linear algebra functions implemented in CBLAS. So my first step was to create CUDA kernels for each of these functions so that I could parallelize the code on GPUs. After, setting up the memory allocation at the beginning of the file and memory freeing at the end I was ready to start parallelizing.

2 The power of parallelization

After using `nvprof` it became clear that the slowest functions were `cuda_daxpy` and `cuda_dgemv` which are my implementations of the `cblas_daxpy` and `cblas_dgemv` respectively. Firstly, I modified the `main` function so that it would take the number of threads per block as an input.
Parallelization of `cuda_daxpy`: This parallelization was very straight forward. Since this function adds two vectors, all I did was assign one thread per dimension of the vectors and have that thread perform sum and scalar multiplication only for this assigned dimension.
Parallelization of `cuda_dgemv`: In this case, to perform matrix vector multiplication and vector addition, I assigned each thread to a row of the matrix so that the thread was responsible only for matrix multiplication and vector addition of that row.
After timing the conjugate gradient function for various values of the number of threads per block I was left with the plot show in figure Figure 1.

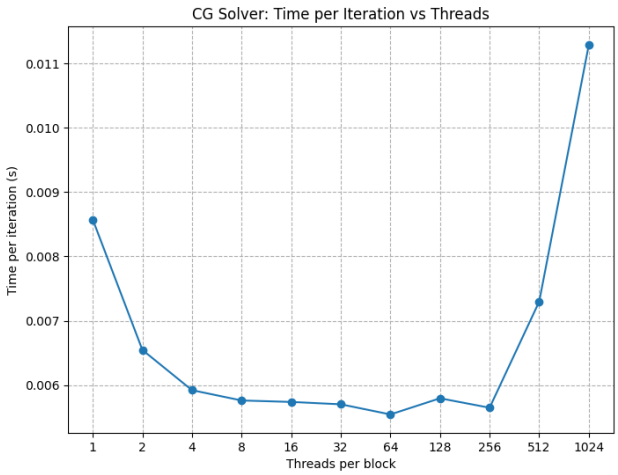

Figure 1: Time per iteration of the conjugate gradient method for varying values of number of threads per block

From the image it is clear that increasing the number of threads per block lowers the time a lot in the beginning until it starts plateauing after 8 threads per block. The worst part however is that after we reach 256 threads per block the times actually start increasing. This effect is so prominent that with 1024 threads per block the time per iteration is actually larger than if we had used 1 thread per block. This behaviour is likely caused by memory bottlenecks. Since 1024 is the maximum threads per block on Izar GPUs, the global memory might not have enough bandwidth to send and/or receive enough data to/from all the threads at the same time. This will lead to a lot of threads waiting for data rather than performing computations. As the matrix we are working with is large it is not possible to eliminate this effect by moving the entire matrix onto the shared memory. It was thus clear that the problem had become memory bounded, nevertheless, my attempts of mitigating this problem were unsuccessful.

3 Conclusion and further considerations

Unfortunately, I could not find a remedy for the problem explained above. I tried other ways of parallelizing or different memory allocations but I did not find a solution that solved the issue. At this point it is clear that the bottleneck is the `cuda_dgemv`, and actually, by inspection of the `nvprof` results I noticed that steep increase in time per iteration was entirely driven by the `cuda_dgemv` function.